

# SMART AI MONITORING - DETAILED DOCUMENTATION

## SMART AI MONITORING DASHBOARD: DEEP-DIVE DOCUMENTATION

This document provides a comprehensive analysis and operational guide for the **Smart AI Monitoring Dashboard**, a multi-component surveillance and automated attendance system.

---

### Project Fundamentals

The engine is built on **Python/Flask** and integrates two high-frequency concurrent models (Fire & Face Recognition) and two event-based models (Object/Weapon & Vehicle).

### System Requirements

- **Operating System**: Windows (optimally), Linux.
  - **Python**: 3.8 - 3.11.
  - **Hardware Accelerator**: CUDA-compatible GPU (recommended for YOLOv8) or high-performance CPU for YuNet/SFace.
- 

## Core Components & Algorithms

### 1. Advanced Fire Detection

The system doesn't rely on simple color-thresholding. It uses a **Heuristic-AI Hybrid Approach**:

- **Dynamic Background Subtraction**: Identifies motion within flame-colored regions.
- **Blink Detection**: Monitors the flickering frequency of incandescent areas to distinguish fire from stationary red objects (like fire extinguishers or signage).
- **Automatic Snapshot API**: Whenever fire is detected with a confidence  $> 0.70$ , a timestamped frame is saved to `static/screenshots/``.

### 2. Facial Recognition & Attendance Loop

Powered by **OpenCV's YuNet** (Detection) and **SFace** (Recognition):

- **Sequence**: Detection (640x480)  $\rightarrow$  Landmark Alignment  $\rightarrow$  128-D Vector Encoding  $\rightarrow$  Cosine Similarity Matching.
- **Check-In/Out Logic**:

# SMART AI MONITORING - DETAILED DOCUMENTATION

- **Check-In**: Triggered on the first high-confidence match ( $\text{Similarity} > 0.65$ ) within the calendar day.
- **Check-Out**: Automatically updated on the *last* known sighting of the user for that day.
- **Anti-Flicker Algorithm**: The system maintains a `latest_detections` cache to prevent "shimmering" boxes when a person is between frames.

## 3. Security Threat Detection (YOLOv8)

- **Models**: Loads `yolov8n.pt` for real-time inference.
- **Detected Objects**: Weapons (knives, pistols), mobile phones (optional), and hazardous tools.
- **Spatial Awareness**: Integrates with the fire detector to alert if a person/weapon is within the proximity of a detected fire hazard.

## 4. Vehicle & Traffic Intelligence

- **Helmet Detection**: A fine-tuned `helmet_best.pt` model analyzes riders.
  - **Number Plate OCR**: Combines YOLO-based plate localization with OCR to extract and log alphanumeric characters to `database/vehicle_logs`.
- 

# Technical Architecture

## Concurrent Threading Model

The application separates the Web interface from the Detection engine to ensure 0% latency on the user dashboard.

1. **Main Flask Thread**: Handles HTTP requests, session management (Flask-Session), and API endpoints.
2. **Camera Supervisor (`camera_supervisor_loop`)**: Monitors all defined camera sources (USB/RTSP) and ensures their capture threads are healthy.
3. **Detection Thread (`detection_loop`)**: Pulsating AI loop that iterates through all frames, performing inference and updating the global `camera_statuses`.
4. **MJPEG Stream Controller**: Distributes the "processed" frames (with bboxes and labels) to multiple concurrent web viewers.

## Project Directory Mapping

- `/detection_models`: Contains the four core AI handlers (`fire_detection.py`, `face_attendance.py`,

# SMART AI MONITORING - DETAILED DOCUMENTATION

`object\_detector.py`, `vehicle\_detector.py`).

- `/camera\_manager`: Encapsulates OpenCV interaction logic and resolution scaling.
  - `/static/js/main.js`: Handles real-time DOM updates and AJAX polling for the dashboard metrics.
- 

## Deployment & Administration

### Adding a New User

1. Navigate to the **Face Registration** page.
2. Assign a name and capture a portrait image.
3. The system will automatically encode the image and store the fingerprint in the `face\_fingerprints` database.

### Modifying Camera Sources

Cameras are managed via the `cameras` dashboard. You can add:

- **Internal Webcam**: Index `0`, `1`, etc.
  - **IP Cameras**: `rtsp://username:password@ip\_address:port/path`.
- 

### Future Enhancements

- **Cloud Sync**: Automatic backup of attendance CSVs to Google Sheets/OneDrive.
  - **SMS Integration**: Instant SMS/WhatsApp alerts on fire or weapon detection.
  - **Edge Optimization**: Quantizing YOLO models for use on Raspberry Pi/Jetson Nano.
- 

- 2024 Smart AI Monitoring Systems - Confidential Project Documentation\*